# 第 3 章 数据结构

## 1. 单调栈

### 1. 什么是单调栈?

单调栈，顾名思义，就是具有单调性的栈。它依旧是一个栈结构，只不过里面存储的数据是递增或者递减的。这种结构是很容易实现的（如下面的代码），但重点是维护一个单调栈的意义是什么?

```cpp
#include <iostream>
#include <stack>

using namespace std;
const int N = 3e6 + 10;
int a[N], n;

void test1()
{
    stack<int> st; // 维护一个单调递增的栈
    for(int i = 1; i <= n; i++)
    {
        // 栈里面大于等于 a[i] 的元素全部出栈
        while(st.size() && st.top() >= a[i]) st.pop();
        st.push(a[i]);
    }
}

void test2()
{
    stack<int> st; // 维护一个单调递减的栈
    for(int i = 1; i <= n; i++)
    {
        // 栈里面小于等于 a[i] 的元素全部出栈
        while(st.size() && st.top() <= a[i]) st.pop();
        st.push(a[i]);
    }
}
```

### 2. 单调栈解决的问题

单调栈能帮助我们解决以下四个问题：

- 寻找当前元素左侧，离它最近，并且比它大的元素在哪；
- 寻找当前元素左侧，离它最近，并且比它小的元素在哪；
- 寻找当前元素右侧，离它最近，并且比它大的元素在哪；
- 寻找当前元素右侧，离它最近，并且比它小的元素在哪。

虽然是四个问题，但是原理是一致的。因此，只要解决一个，举一反三就可以解决剩下的几个。

### 3. 寻找当前元素左侧，离它最近，并且比它大的元素在哪

> 从左往右遍历元素，构造一个单调递减的栈。插入当前位置的元素的时：
>
> - 如果栈为空，则左侧不存在比当前元素大的元素；
> - 如果栈非空，插入当前位置元素时的栈顶元素就是所找的元素。
>
> 注意，因为我们要找的是最终结果的位置。因此，栈里面存的是每个元素的下标。

**【测试用例】**

```
输入：
9
1 4 10 6 3 3 15 21 8

输出：
0 0 0 3 4 4 0 0 8
```

**【代码实现】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

const int N = 3e6 + 10;

int a[N], n;
int ret[N];

void test()
{
    stack<int> st; // 维护一个单调递减的栈
```

```cpp

    for(int i = 1; i <= n; i++)
    {
        // 栈里面小于等于 a[i] 的元素全部出栈
        while(st.size() && a[st.top()] <= a[i]) st.pop();

        // 此时栈顶元素存在，栈顶元素就是所求结果
        if(st.size()) ret[i] = st.top();

        st.push(i); // 存的是下标
    }

    for(int i = 1; i <= n; i++)
    {
        cout << ret[i] << " ";
    }
    cout << endl;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    test(); cout << endl;

    return 0;
}
```

### 4. 寻找当前元素左侧，离它最近，并且比它小的元素在哪

> 从左往右遍历元素，构造一个单调递增的栈。插入当前位置的元素的时：
>
> - 如果栈为空，则左侧不存在比当前元素小的元素；
> - 如果栈非空，插入当前位置元素时的栈顶元素就是所找的元素。
>
> 注意，因为我们要找的是最终结果的位置。因此，栈里面存的是每个元素的下标。

**【测试用例】**

```
输入：
9
1 4 10 6 3 3 15 21 8
```

```
输出:
0 1 2 2 1 1 6 7 6
```

**【代码实现】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

const int N = 3e6 + 10;

int a[N], n;
int ret[N];

void test()
{
    stack<int> st; // 维护一个单调递增的栈

    for(int i = 1; i <= n; i++)
    {
        // 栈里面大于等于 a[i] 的元素全部出栈
        while(st.size() && a[st.top()] >= a[i]) st.pop();

        // 此时栈顶元素存在，栈顶元素就是所求结果
        if(st.size()) ret[i] = st.top();

        st.push(i); // 存的是下标
    }

    for(int i = 1; i <= n; i++)
    {
        cout << ret[i] << " ";
    }
    cout << endl;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    test(); cout << endl;
```

```
39
40      return 0;
41  }
```

针对其余两种情况，我们仅需逆序遍历数组即可。

### 5. 寻找当前元素右侧，离它最近，并且比它大的元素在哪

> **从右往左**遍历元素，构造一个单调递减的栈。插入当前位置的元素的时：
>
> - 如果栈为空，则左侧不存在比当前元素大的元素；
> - 如果栈非空，插入当前位置元素时的栈顶元素就是所找的元素。
>
> 注意，因为我们要找的是最终结果的位置。因此，栈里面存的是每个元素的下标。

**【测试用例】**

```
输入：
9
1 4 10 6 3 3 15 21 8

输出：
2 3 7 7 7 7 8 0 0
```

**【代码实现】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

const int N = 3e6 + 10;

int a[N], n;
int ret[N];

void test()
{
    stack<int> st; // 维护一个单调递减的栈

```

```cpp
    for(int i = n; i >= 1; i--)
    {
        // 栈里面小于等于 a[i] 的元素全部出栈
        while(st.size() && a[st.top()] <= a[i]) st.pop();

        // 此时栈顶元素存在，栈顶元素就是所求结果
        if(st.size()) ret[i] = st.top();

        st.push(i); // 存的是下标
    }

    for(int i = 1; i <= n; i++)
    {
        cout << ret[i] << " ";
    }
    cout << endl;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    test(); cout << endl;

    return 0;
}
```

### 6. 寻找当前元素右侧，离它最近，并且比它小的元素在哪

> **从右往左**遍历元素，构造一个单调递增的栈。插入当前位置的元素的时：
>
> - 如果栈为空，则左侧不存在比当前元素小的元素；
> - 如果栈非空，插入当前位置元素时的栈顶元素就是所找的元素。
>
> 注意，因为我们要找的是最终结果的位置。因此，栈里面存的是每个元素的下标。

**【测试用例】**

```
输入：
9
1 4 10 6 3 3 15 21 8

```

```
输出:
0 5 4 5 0 0 9 9 0
```

**【代码实现】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

const int N = 3e6 + 10;

int a[N], n;
int ret[N];

void test()
{
    stack<int> st; // 维护一个单调递增的栈

    for(int i = n; i >= 1; i--)
    {
        // 栈里面大于等于 a[i] 的元素全部出栈
        while(st.size() && a[st.top()] >= a[i]) st.pop();

        // 此时栈顶元素存在，栈顶元素就是所求结果
        if(st.size()) ret[i] = st.top();

        st.push(i); // 存的是下标
    }

    for(int i = 1; i <= n; i++)
    {
        cout << ret[i] << " ";
    }
    cout << endl;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    test(); cout << endl;

```

```
40        return 0;
41    }
```

💡 **总结：**

- 找左侧，正遍历；找右侧，逆遍历；
- 比它大，单调减；比它小，单调增。

## 1.1 【模板】单调栈

**题目来源：** 洛谷

**题目链接：** P5788 【模板】单调栈

**难度系数：** ★★

**【题目描述】**

给出项数为 $n$ 的整数数列 $a_{1...n}$。

定义函数 $f(i)$ 代表数列中第 $i$ 个元素之后第一个大于 $a_i$ 的元素的下标，即 $f(i)=min_{i<j\leq n,aj>ai}j$。若不存在，则 $f(i)=0$。

试求出 $f(1...n)$。

**【输入描述】**

第一行一个正整数 $n$。

第二行 $n$ 个正整数 $a_{1...n}$。

对于 $100\%$ 的数据， $1\leq n\leq 3\times 10^6, 1\leq a_i\leq 10^9$。

**【输出描述】**

一行 $n$ 个整数表示 $f(1),f(2),...,f(n)$ 的值。

**【示例一】**

输入：

```
5
1 4 2 3 5
```

输出：

> 25450

**【解法】**

右侧离它最近并且比它大的元素：

- 逆序遍历数组；
- 构造一个单调递减的栈；
- 进栈时，栈顶元素就是最终结果。

**【参考代码】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

const int N = 3e6 + 10;

int n;
int a[N];
int ret[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    stack<int> st;
    for(int i = n; i >= 1; i--)
    {
        // 单调递减的栈 - 元素的下标
        while(st.size() && a[st.top()] <= a[i]) st.pop();

        if(st.size()) ret[i] = st.top();
        st.push(i);
    }

    for(int i = 1; i <= n; i++) cout << ret[i] << " ";
    cout << endl;

    return 0;
}
```

## 1.2 发射站

> **题目来源：** 洛谷
>
> **题目链接：** P1901 发射站
>
> **难度系数：** ★★

**【题目描述】**

> 某地有 $N$ 个能量发射站排成一行，每个发射站 $i$ 都有不相同的高度 $H_i$，并能向两边（两端的发射站只能向一边）同时发射能量值为 $V_i$ 的能量，发出的能量只被两边最近的且比它高的发射站接收。显然，每个发射站发来的能量有可能被 $0$ 或 $1$ 或 $2$ 个其他发射站所接受。
>
> 请计算出接收最多能量的发射站接收的能量是多少。

**【输入描述】**

> 第 $1$ 行一个整数 $N$。
>
> 第 $2$ 到 $N+1$ 行，第 $i+1$ 行有两个整数 $H_i$ 和 $V_i$，表示第 $i$ 个发射站的高度和发射的能量值。

**【输出描述】**

> 输出仅一行，表示接收最多能量的发射站接收到的能量值。答案不超过 32 位带符号整数的表示范围。

**【示例一】**

> 输入：
>
> ```
> 3
> 4 2
> 3 5
> 6 10
> ```
>
> 输出：
>
> ```
> 7
> ```

**【解法】**

有了单调栈之后，这道题就变成模拟题了.....

**【参考代码】**

```
1  #include <iostream>
2  #include <stack>
```

```cpp

using namespace std;

typedef long long LL;

const int N = 1e6 + 10;

int n;
LL h[N], v[N];
LL sum[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> h[i] >> v[i];

    // 找左边
    stack<int> st;
    for(int i = 1; i <= n; i++)
    {
        // 单调递减的栈 - 存下标
        while(st.size() && h[st.top()] <= h[i]) st.pop();
        if(st.size())
        {
            sum[st.top()] += v[i];
        }
        st.push(i);
    }

    // 找右边
    while(st.size()) st.pop(); // 清空
    for(int i = n; i >= 1; i--)
    {
        // 单调递减的栈 - 存下标
        while(st.size() && h[st.top()] <= h[i]) st.pop();
        if(st.size())
        {
            sum[st.top()] += v[i];
        }
        st.push(i);
    }

    LL ret = 0;
    for(int i = 1; i <= n; i++) ret = max(ret, sum[i]);

    cout << ret << endl;

```

```
50        return 0;
51    }
```

## 1.3 Largest Rectangle in a Histogram

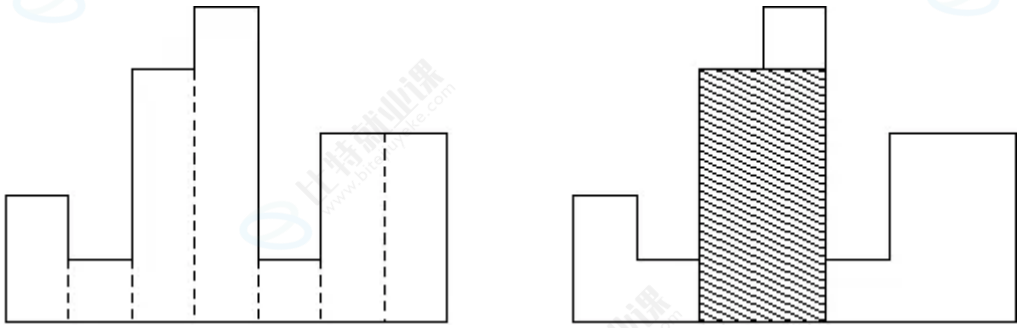
**题目来源：** 洛谷

**题目链接：** HISTOGRA - Largest Rectangle in a Histogram

**难度系数：** ★★★

**【题目描述】**

如图所示，在一条水平线上有 $n$ 个宽为 $1$ 的矩形，求包含于这些矩形的最大子矩形面积（图中的阴影部分的面积即所求答案）。



**【输入描述】**

有多组测试数据，每组数据占一行。输入零时读入结束。

每行开头为一个数字 $n(1 \leq n \leq 10^5)$，接下来在同一行给出 $n$ 个数字 $h_1, h_2, \cdots, h_n(0 \leq h_i \leq 10^9)$，表示每个矩形的高度。

**【输出描述】**

对于每组数据，输出最大子矩阵面积，一组数据输出一行。

**【示例一】**

输入：

```
7 2 1 4 5 1 3 3
4 1000 1000 1000 1000
0
```

输出：

```
8
4000
```

**【解法】**

对于 $x$ 位置子矩阵，找到左侧离它最近并且比它小的位置 $y$ ，那么 $[x+1, y]$ 之间就是该矩阵能到达的左端。

同理再找到右侧离它最近并且比它小的位置 $z$ ，那么 $[y, z-1]$ 之间就是该矩阵能到达的右端。

对于每一个子矩阵，求出它向左以及向右能延伸的最大长度即可。

**【参考代码】**

```cpp
#include <iostream>
#include <stack>

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

int n;
LL h[N];
LL x[N], y[N];

int main()
{
    while(cin >> n, n)
    {
        for(int i = 1; i <= n; i++) cin >> h[i];

        // 找左边，小
        stack<int> st;
        for(int i = 1; i <= n; i++)
        {
            // 单调递增的栈 - 存下标
            while(st.size() && h[st.top()] >= h[i]) st.pop();

            if(st.size()) x[i] = st.top();
            else x[i] = 0;

            st.push(i);
        }

        // 找右边，小
        while(st.size()) st.pop();

        for(int i = n; i >= 1; i--)
        {
```

```
38              // 单调递增的栈 - 存下标
39              while(st.size() && h[st.top()] >= h[i]) st.pop();
40
41              if(st.size()) y[i] = st.top();
42              else y[i] = n + 1;
43
44              st.push(i);
45          }
46
47          LL ret = 0;
48          for(int i = 1; i <= n; i++)
49          {
50              ret = max(ret, h[i] * (y[i] - x[i] - 1));
51          }
52
53          cout << ret << endl;
54      }
55
56      return 0;
57  }
```

## 2. 单调队列

### 1. 什么是单调队列?

单调队列，顾名思义，就是存储的元素要么单调递增要么单调递减的队列。注意，这里的队列和普通的队列不一样，是一个双端队列。

### 2. 单调队列解决的问题

一般用于解决滑动窗口内最大值最小值问题，以及优化动态规划。

## 2.1 【模板】单调队列

> **题目来源：** 洛谷
>
> **题目链接：** P1886 滑动窗口 /【模板】单调队列
>
> **难度系数：** ★★

**【题目描述】**

> 有一个长为 $n$ 的序列 $a$ ，以及一个大小为 $k$ 的窗口。现在这个从左边开始向右滑动，每次滑动一个单位，求出每次滑动后窗口中的最大值和最小值。

例如，对于序列 $[1,3,-1,-3,5,3,6,7]$ 以及 $k=3$，有如下过程：

| 窗口位置 | 最小值 | 最大值 |
| --- | --- | --- |
| [1 3 -1] -3 5 3 6 7 | $-1$ | 3 |
| 1 [3 -1 -3] 5 3 6 7 | $-3$ | 3 |
| 1 3 [-1 -3 5] 3 6 7 | $-3$ | 5 |
| 1 3 -1 [-3 5 3] 6 7 | $-3$ | 5 |
| 1 3 -1 -3 [5 3 6] 7 | 3 | 6 |
| 1 3 -1 -3 5 [3 6 7] | 3 | 7 |

**【输入描述】**

输入一共有两行，第一行有两个正整数 $n,k$。 第二行 $n$ 个整数，表示序列 $a$

**【输出描述】**

输出共两行，第一行为每次窗口滑动的最小值

第二行为每次窗口滑动的最大值

**【示例一】**

输入：

```
8 3
1 3 -1 -3 5 3 6 7
```

输出：

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

**【解法】**

窗口内最大值：

从左往右遍历元素，维护一个单调递减的队列：

- 当前元素进队之后，注意维护队列内的元素在大小为 k 的窗口内；
- 此时队头元素就是最大值。

窗口内最小值：

从左往右遍历元素，维护一个单调递增的队列：

- 当前元素进队之后，注意维护队列内的元素在大小为 k 的窗口内；

- 此时队头元素就是最小值。

**【参考代码】**

```cpp
#include <iostream>
#include <deque>

using namespace std;

const int N = 1e6 + 10;

int n, k;
int a[N];

int main()
{
    cin >> n >> k;
    for(int i = 1; i <= n; i++) cin >> a[i];

    deque<int> q; // 存下标
    // 窗口内最小值 - 单调递增的队列 - 存下标
    for(int i = 1; i <= n; i++)
    {
        while(q.size() && a[q.back()] >= a[i]) q.pop_back();

        q.push_back(i);

        // 判断队列里面元素是否在合法窗口内
        if(q.back() - q.front() + 1 > k) q.pop_front();

        if(i >= k) cout << a[q.front()] << " ";
    }
    cout << endl;

    // 窗口内最大值 - 单调递减的队列 - 存下标
    q.clear();
    for(int i = 1; i <= n; i++)
    {
        while(q.size() && a[q.back()] <= a[i]) q.pop_back();

        q.push_back(i);

        if(q.back() - q.front() + 1 > k) q.pop_front();
        if(i >= k) cout << a[q.front()] << " ";
    }
```

```
42        cout << endl;
43
44        return 0;
45    }
```

## 2.2 质量检测

**题目来源：** 洛谷

**题目链接：** P2251 质量检测

**难度系数：** ★★

**【题目描述】**

为了检测生产流水线上总共 $N$ 件产品的质量，我们首先给每一件产品打一个分数 $A$ 表示其品质，然后统计前 $M$ 件产品中质量最差的产品的分值 $Q[m]=min\{A_1,A_2,...A_m\}$，以及第 $2$ 至第 $M+1$ 件的 $Q[m+1],Q[m+2]...$ 最后统计第 $N-M+1$ 至第 $N$ 件的 $Q[n]$。根据 $Q$ 再做进一步评估。

请你尽快求出 $Q$ 序列。

**【输入描述】**

输入共两行。

第一行共两个数 $N$、$M$，由空格隔开。含义如前述。

第二行共 $N$ 个数，表示 $N$ 件产品的质量。

**【输出描述】**

输出共 $N-M+1$ 行。

第 $1$ 至 $N-M+1$ 行每行一个数，第 $i$ 行的数 $Q[i+M-1]$。含义如前述。

**【示例一】**

输入：

10 4

16 5 6 9 5 13 14 20 8 12

输出：

5

5

5

5

```
5
8
8
```

**【解法】**

滑动窗口内的最小值~

**【参考代码】**

```cpp
#include <iostream>
#include <deque>

using namespace std;

const int N = 1e6 + 10;

int n, k;
int a[N];

int main()
{
    cin >> n >> k;

    // 窗口内最小值
    deque<int> q; // 单调递增
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i];

        // 维护递增
        while(q.size() && a[q.back()] >= a[i]) q.pop_back();

        q.push_back(i);

        // 维护队列内元素合法
        if(i - q.front() + 1 > k) q.pop_front();

        // 输出结果
        if(i >= k) cout << a[q.front()] << endl;
    }

    return 0;
}
```